

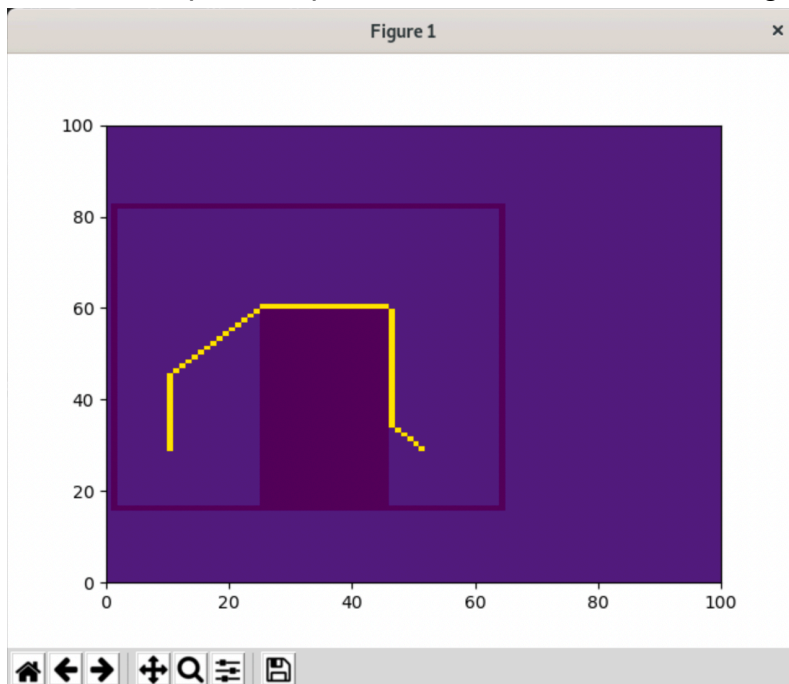
Aidan Rohm - Midterm Exam - Professor Lyons - 3.19.26

Midterm Exam Submission for Artificial Intelligence

Question 1:

Part A:

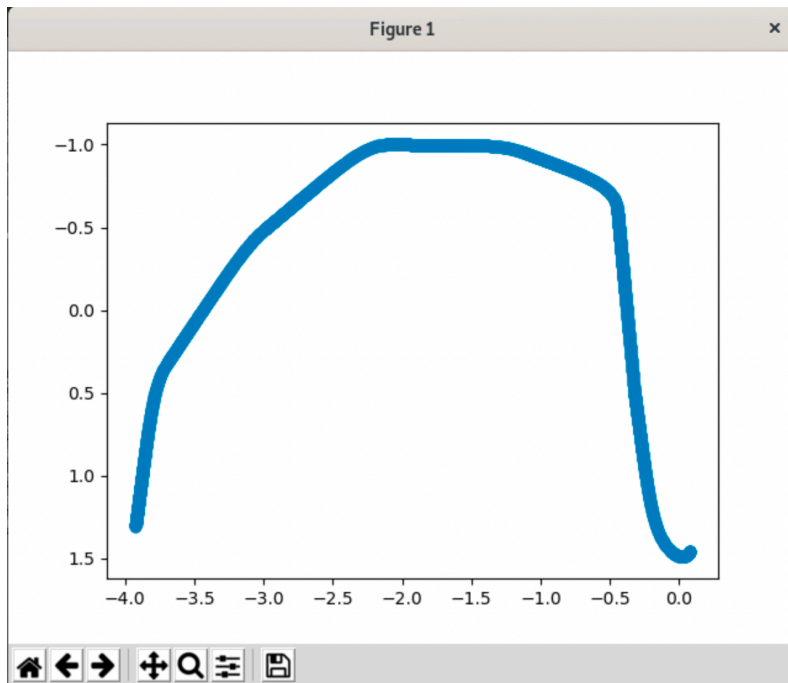
Below is the planned path of the robot after conducting an A-Star search for the goal node.



Below is the output from the program demonstrating the number of nodes that were expanded in order to find the goal.

```
Path planning from: (70.0, 51.0) to (70.0, 10.0)
Found goal (70.0, 10.0)
Nodes expanded: 916
```

Below is the actual path of the robot after following the path that it planned using A-Star.



Part B:

Pseudocode for bidirectional A-Star search algorithm:

```
# Based on the bidirectional A* logic described below
function bidirectionalAstar(map, start, goal):

    # Create the forward frontier starting from the start node
    # Each entry stores: current location, g-cost so far, and path from
    start
    forward_frontier = [(start, 0, [start])]

    # Create the backward frontier starting from the goal node
    # Each entry stores: current location, g-cost so far, and path from goal
    backward_frontier = [(goal, 0, [goal])]

    # Create closed lists for the two search directions
    forward_closed = empty set
    backward_closed = empty set

    # Store the best node seen so far at each location in each direction
    forward_best = {start: (0, [start])}
    backward_best = {goal: (0, [goal])}

    # Continue searching while both frontiers still contain nodes
    while forward_frontier is not empty AND backward_frontier is not empty:
```

```

# Sort the forward frontier using the forward A* evaluation
#  $f(n) = g(n) + h(n, \text{goal})$ 
sort forward_frontier by:
    f_forward(n) = g_forward(n) + heuristic(location, goal)

# Remove the best forward node from the frontier
current_forward = remove first node from forward_frontier
(f_loc, f_g, f_path) = current_forward

# If this forward node has already been reached by the backward
search,
# then the two searches have met and a full path can be returned
if f_loc is in backward_closed OR f_loc is in backward_best:
    backward_path = path stored for f_loc in backward_best
    join f_path with backward_path at f_loc
    return complete joined path

# Mark this location as explored in the forward direction
add f_loc to forward_closed

# Expand all valid successors of the current forward node
forward_successors = successor(map, f_loc)
for each successor in forward_successors:
    successor_loc = successor location
    step_cost = cost of moving to successor
    new_g = f_g + step_cost

    # Only keep this successor if it is new,
    # or if it is cheaper than the old path to that location
    if successor_loc is not in forward_best
        OR new_g < stored g-cost in forward_best:

        new_path = f_path + [successor_loc]
        store (new_g, new_path) in forward_best at successor_loc
        add (successor_loc, new_g, new_path) to forward_frontier

# Sort the backward frontier using the backward A* evaluation
#  $f(n) = g(n) + h(n, \text{start})$ 
sort backward_frontier by:
    f_backward(n) = g_backward(n) + heuristic(location, start)

# Remove the best backward node from the frontier
current_backward = remove first node from backward_frontier
(b_loc, b_g, b_path) = current_backward

# If this backward node has already been reached by the forward

```

```

search,
    # then the two searches have met and a full path can be returned
    if b_loc is in forward_closed OR b_loc is in forward_best:
        forward_path = path stored for b_loc in forward_best
        join forward_path with b_path at b_loc
        return complete joined path

    # Mark this location as explored in the backward direction
    add b_loc to backward_closed

    # Expand all valid successors of the current backward node
    backward_successors = successor(map, b_loc)
    for each successor in backward_successors:
        successor_loc = successor location
        step_cost = cost of moving to successor
        new_g = b_g + step_cost

        # Only keep this successor if it is new,
        # or if it is cheaper than the old path to that location
        if successor_loc is not in backward_best
            OR new_g < stored g-cost in backward_best:

            new_path = b_path + [successor_loc]
            store (new_g, new_path) in backward_best at successor_loc
            add (successor_loc, new_g, new_path) to backward_frontier

    # If the two searches never meet, then no path was found
    return failure

```

Explanation:

Bidirectional A-Star runs two heuristic searches at once. The **forward search** starts at the robot's start position and uses:

$$f_f(n) = g_f(n) + h_f(n)$$

where:

$$g_f(n)$$

- is the cost from the goal to node n and,

$$h_f(n)$$

- is the straight line distance from n to the goal.

The **backward search** starts at the goal and uses:

$$f_b(n) = g_b(n) + h_b(n)$$

where:

$$g_b(n)$$

- is the cost from the goal to node n and,

$$h_b(n)$$

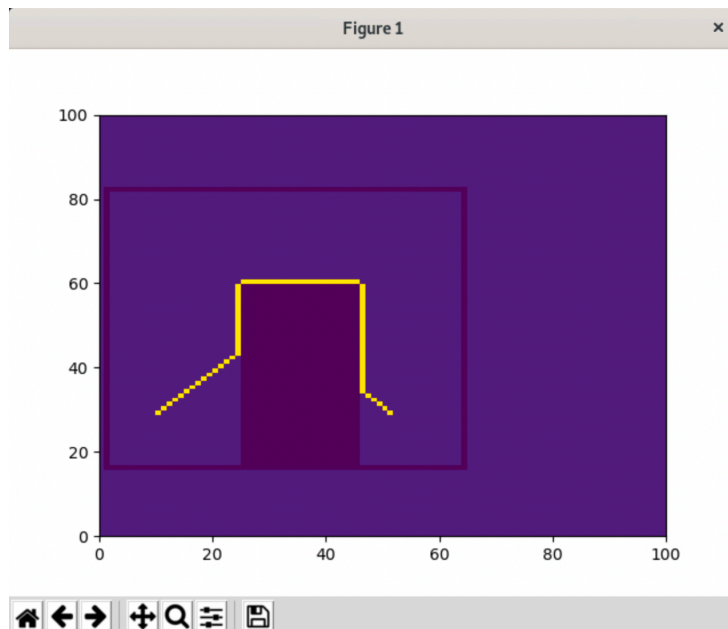
- is the straight line distance from n back to the start

Instead of searching for the entire path from the starting location, a bidirectional A-Star algorithm searches from the start node and the end node, hoping to meet at a node in the middle. In theory, when a node is reached in one direction it is also reached by the other direction. From that node, we can combine the two paths:

- One path from the start to the meeting node
- One path from the meeting node to the goal (by reversing this "side" of the searches current path).

This will give the final route. A bidirectional A-Star is usually faster than a normal A-Star because each "side" of the algorithm searches less of the map before the searches meet.

Below is the planned path of the robot after conducting a bidirectional A-Star search for the goal node.

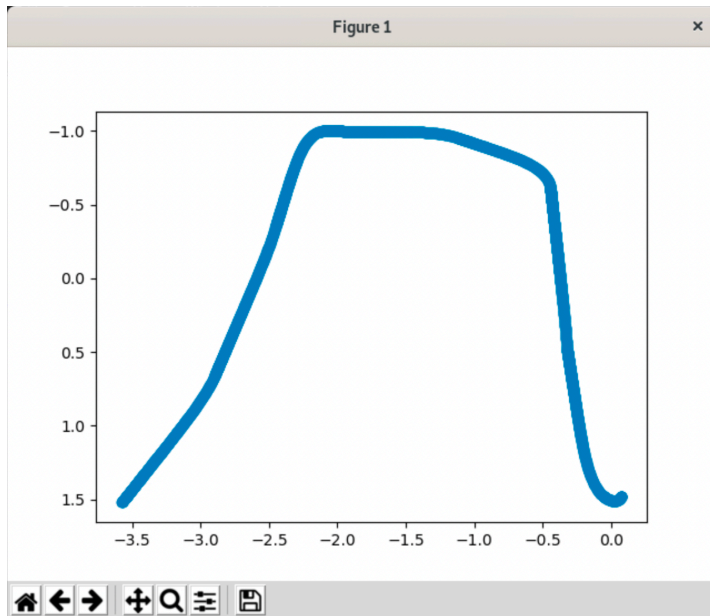


Below is the output from the program demonstrating the number of nodes that were expanded in order to find the goal, including a number from both the forward and backward expansions.

```
Path planning from: (70.0, 51.0) to (70.0, 10.0)
Searches met at (39.0, 25.0)
Forward nodes expanded: 871
Backward nodes expanded: 871
```

Below is the actual path of the robot after following the path that it planned using bidirectional A-

Star.



PLEASE NOTE: the submitted code for the A-Star and bidirectional A-Star have been titled `rohm_astar.py` and `rohm_astarbidirectional.py` respectively. This was done to remain consistent with the syllabuses outline for submitting python files.

Question 2

Part A:

The board array contains 'X', 'O', or ' ' at each position within the board. The board starts completely empty, meaning that all squares on the board are blank. Therefore, every individual square on the board is represented by a ' '. This also means that `X_ij` and `O_ij` at every position are false until players actually begin to make moves. The simple model to depict any state of the board relied on the following logic:

```
For each cell (i, j) on the board:
    if board[i][j] == 'X':
        X_ij = True
    if board[i][j] == 'O':
        O_ij = True
    if board[i][j] == ' ':
        X_ij and O_ij both remain false
```

Take the following example:

```
'X'|'O'|' '|
---|---|---
'|'|'X'|'O'
```

```

---|---|---
'X'|'|'|'

```

The model mapping will bring:

True propositions:

X_00, X_11, X_20, O_01, and O_12

False propositions (empty cells):

All other X_{ij} and O_{ij}

Now, generally speaking the board state can be represented as a conjunction of these different values.

Part B:

The problem defines that C is a win condition where C can be a winning configuration of Xs or Os. Below, are just a few different examples of winning conditions:

X X X	0	X
--- --- ---	--- --- ---	--- --- ---
	0	X
--- --- ---	--- --- ---	--- --- ---
	0	X

For this general logical statement, Xs and Os are replaced by Cs with corresponding locations. In a game of Tic-Tac-Toe, there are 16 possible winning orientations, 8 orientations for X and 8 orientations for O. Therefore, the propositional logic statement that describes every win condition for C is as follows:

$$\begin{aligned}
 \text{Win}(C) \equiv & (C_{00} \wedge C_{01} \wedge C_{02}) \vee (C_{10} \wedge C_{11} \wedge C_{12}) \vee (C_{20} \wedge C_{21} \wedge C_{22}) \vee \\
 & (C_{00} \wedge C_{10} \wedge C_{20}) \vee (C_{01} \wedge C_{11} \wedge C_{21}) \vee (C_{02} \wedge C_{12} \wedge C_{22}) \vee \\
 & (C_{00} \wedge C_{11} \wedge C_{22}) \vee (C_{02} \wedge C_{11} \wedge C_{20})
 \end{aligned}$$

It is important to note that this does NOT represent all possible states of the board, this is only highlighting a winning condition defined by the rules where the X or O player needs to get 3 of their symbols in a line.

Part C:

The following pseudocode is developed from the idea that a third "ply" is necessary to check whether or not there is a forced or inevitable victory. The questions was taken with the idea that ply 1 checks whether or not there is a forked position for a player to take. In this case, a win is inevitable because no matter what move is taken in the second ply, minimax will choose the closing move on the third ply. The third ply lookup is essential to evaluate whether or not an inevitable win actually occurs after two ply.

```

# Checks whether the current player has an inevitable win
# within 2 ply from the current board:
#   ply 1 = current player moves
#   ply 2 = opponent moves

def win_inevitable_2ply(board, player):

    if player == 'X':
        opponent = 'O'
    else:
        opponent = 'X'

    # Try every legal move for the current player (ply 1)
    for each cell (i, j) in get_empty_cells(board):

        board_after_player = place(board, i, j, player)
        model_after_player = build_model(board_after_player)

        # If the current player wins immediately after ply 1,
        # this is already a forced win
        if evaluate_Win(model_after_player, player):
            return True

        # Assume this move is forcing unless an opponent reply breaks it
        player_forces_win = True

        # Try every legal response by the opponent (ply 2)
        for each cell (r, c) in get_empty_cells(board_after_player):

            board_after_opponent = place(board_after_player, r, c, opponent)
            model_after_opponent = build_model(board_after_opponent)

            # If the opponent wins immediately after ply 2,
            # this move is not a forced win
            if evaluate_Win(model_after_opponent, opponent):
                player_forces_win = False
                break

        # After this opponent reply, the current player must still
        # have some winning next move, otherwise the original move
        # was not inevitable
        player_has_winning_reply = False

        # This is essentially checking a third ply
        # Does not change the inevitability, but needs to know that a
        # winning move is still available

```



```

        for each cell (x, y) in get_empty_cells(board_after_opponent):

            board_after_reply = place(board_after_opponent, x, y,
player)

            model_after_reply = build_model(board_after_reply)

            if evaluate_Win(model_after_reply, player):
                player_has_winning_reply = True
                break

            if not player_has_winning_reply:
                player_forces_win = False
                break

        # If every opponent reply still allows the current player
        # to win on the following move, then this move is inevitable
        if player_forces_win:
            return True

    # There is no inevitable win
    return False

# Constructed from the model proposition from Part A
function build_model(board):
    model = {}
    for each cell (i, j):
        model[X_ij] = (board[i][j] == 'X')
        model[O_ij] = (board[i][j] == 'O')
    return model

# General win evaluator using the same winning pattern from Part B
function evaluate_Win(model, C):
    return (model[C_00] AND model[C_01] AND model[C_02])    # Row 0
           OR (model[C_10] AND model[C_11] AND model[C_12])    # Row 1
           OR (model[C_20] AND model[C_21] AND model[C_22])    # Row 2
           OR (model[C_00] AND model[C_10] AND model[C_20])    # Column 0
           OR (model[C_01] AND model[C_11] AND model[C_21])    # Column 1
           OR (model[C_02] AND model[C_12] AND model[C_22])    # Column 2
           OR (model[C_00] AND model[C_11] AND model[C_22])    # Diagonal
           OR (model[C_02] AND model[C_11] AND model[C_20])    # Anti-diagonal

```

The above pseudocode creates a function that checks for win inevitability after two ply. Additionally, pseudocode is outlined for the two functions that relate to creating a model of the board and checking this model for a win condition. These two functions were already provided

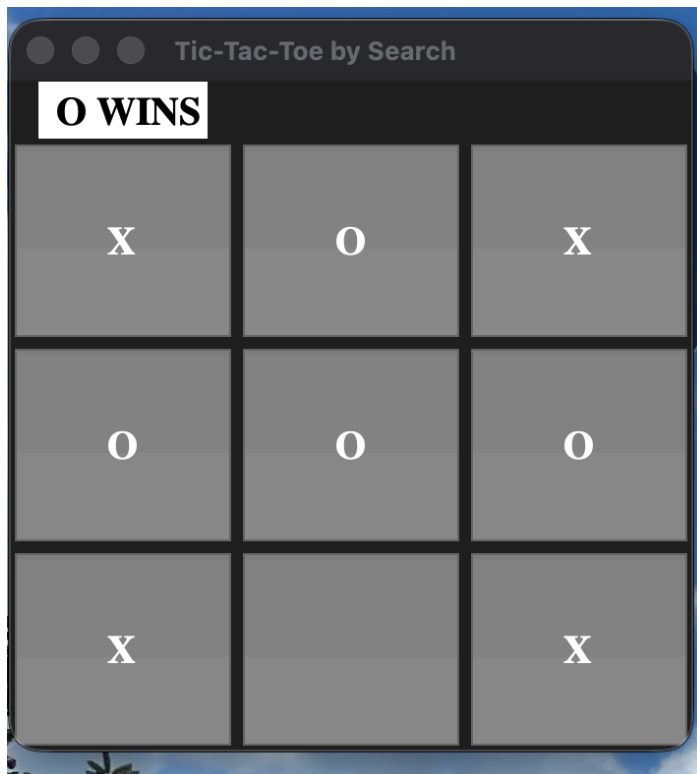
by the minimax code from class, but outlining them anyway was helpful for understanding. The two ply function accurately looks for any move that can be made in the first ply. If this move is a winning one, then obviously the player had an inevitable win within two ply. It then checks every legal response by the opponent as the second ply. The third ply is baked into this second loop in order to check the inevitability of the win. If there is a winning move found, the function returns true and exits. If no winning move is found here, the function returns false and continues normal play.

Part D:

Exists within the minimaxMC.py file. This file has been named rohm_minimaxMC.py to remain consistent with the syllabus and file naming requirements.

Part E:

The following image depicts the Tic-Tac-Toe board that was used to demonstrate the difference between the regular minimax algorithm and the 2-ply minimax algorithm that was implemented.



This board was the result of the following moves:

X -> (0,0)

O -> (1,1)

X -> (2,2)

O -> (0,1)

X -> (0,2)

O -> (1,2)

It is at this point that minimax has a "fork" and can move to either (1,0) or (2,1) and win. This is

when a win is inevitable.

It does not matter where X decides to go from here because no matter what, O will win.

X -> (2,0)

The following is the output from the code where the first running is when regular minimax was used and the second is minimaxMC.

```
• (base) Aidans-MacBook-Pro-2:Question2 aidan$ python3 ttt.py
Nodes expanded in this move: 31972
Minimax chooses (1, 1)
Nodes expanded in this move: 532
Minimax chooses (0, 1)
Nodes expanded in this move: 16
Minimax chooses (1, 2)
Nodes expanded in this move: 0
Minimax chooses (1, 0)
0 wins
Average total nodes expanded per move in game: 8130.0
• (base) Aidans-MacBook-Pro-2:Question2 aidan$ python3 ttt.py
Nodes expanded in this move: 31972
Minimax chooses (1, 1)
Nodes expanded in this move: 532
Minimax chooses (0, 1)
2-ply check: 0 has an inevitable win. Choosing move: (1, 2)
Nodes expanded in this move: 0
Minimax chooses (1, 2)
2-ply check: 0 has an inevitable win. Choosing move: (1, 0)
Nodes expanded in this move: 0
Minimax chooses (1, 0)
0 wins
Average total nodes expanded per move in game: 8126.0
```

The output here demonstrates that after 2 moves from both players, the algorithm was able to find an inevitable win for O in 2 ply. It starts by taking the first move in this process (1,2) as it is still the optimal choice. gNodes is incremented inside maxval() and minval(), and since the 2-ply check stops before those functions are called, gNodes does not get incremented and stays at 0 for this move. The 2-ply found a forcing move using its own lightweight loop and returned directly completely bypassing the recursive (and therefore slow) aspect of minimax.

There are some important things to highlight from the output of the two programs. Of course, minimax made the same moves in each game regardless of which program was actually run. From the output of the modified minimax, we can clearly see when the 2-ply has detected an inevitable win and it was a result of a "fork" from O. Once X moves, and no matter where X moves, there is still at least 1 move for O to make that will secure the win. Once again, this means that gNodes is not incremented because the 2-ply function detects the win before allowing incrementation.

An important point of comparison is the actual impact of the number of nodes expanded. We can see that the first two steps of minimax in both games result in the same number of expanded nodes. These are the largest number of expansions because of the nature of the game at this point. With an empty board and a board with only two occupied spaces, there remains plenty of possible moves and scenarios to check, and none of them guarantee a 2 ply victory. It is not until later in the game that the 2-ply minimax is able to detect a win and cut the search short. Because of this, there is only a 16 node difference between the two search algorithms.

The optimization that was made would show more dramatic savings in a domain where forced wins arise earlier in the search, or in a game that has a larger tree where late-game nodes are still expensive to reach. The nature of Tic-Tac-Toe simply reduces the number of possibilities.